

数据结构期末复习提纲--上&下

1.什么是算法的时间复杂度？大O表示法的真实含义是什么？如何比较两种算法复杂度的大小？

解 如果不考虑变量定义语句,该算法主要包括 3 个可执行语句①、②和③。其中语句①循环控制变量 i 要从 0 增加到 n ,测试 $i=n$ 时才会终止,故它的频度是 $n+1$,但它的循环体却只能执行 n 次。语句②作为语句①循环体内的语句应该只执行 n 次,但语句②本身也要执行 $n+1$ 次,所以语句②的频度是 $n(n+1)$ 。同理可得语句③的频度为 n^2 。因此,该算法中所有语句的频度之和(即执行时间)为

$$T(n) = n + 1 + n(n + 1) + n^2 = 2n^2 + 2n + 1$$

2) $T(n)$ 用“O”表示

由于算法分析不是绝对时间的比较,在求出 $T(n)$ 后,通常进一步采用时间复杂度来表示。算法时间复杂度(time complexity)就是用 $T(n)$ 的数量级来表示,记作 $T(n) = O(f(n))$ 。

在上述表达式中“O”读作“大 O”(是 Order 的简写,意指数量级),其含义是为 $T(n)$ 找到一个上界 $f(n)$,其严格的数学定义是 $T(n)$ 的数量级表示为 $O(f(n))$,是指存在着正常量 c 和 n_0 (为一个足够大的正整数),使得 $\lim_{n \rightarrow n_0} \frac{|T(n)|}{|f(n)|} = c \neq 0$ 成立。所以算法时间复杂度也称为渐进时间复杂度,它表示随问题规模 n 的增大,算法执行时间的增长率和 $f(n)$ 的增长率相同。因此算法时间复杂度分析实际上是一种时间增长趋势分析。

显然, $T(n)$ 的这种上界 $f(n)$ 可能有多个,通常取最紧凑的上界。也就是只求出 $T(n)$ 的最高阶,忽略其低阶项和常系数,这样既可简化 $T(n)$ 的计算,又能比较客观地反映出当 n 很大时算法的时间性能。例如,对于例 1.6 有 $T(n) = 2n^2 + 2n + 1 = O(n^2)$,也就是说,该算法的时间复杂度为 $O(n^2)$ 。

一般情况下,一个没有循环(或者有循环,但循环次数与问题规模 n 无关)的算法中原操作执行次数与问题规模 n 无关,记作 $O(1)$,也称为常数阶。算法中的每个简单语句,例如定义变量语句、赋值语句和输入输出语句,其执行时间都看成是 $O(1)$ 。

算法的时间复杂度是一种对算法运行用时关于输入数据增加时的增长趋势的分析

大O表述法实际上描述的是时间复杂度的数量级 即常数级 $O(1)$ 线性基 $O(n)$

指数级 $O(2^n)$ 对数级 $O(\log n)$

比较复杂度大小往往就是比较数量级之间的大小关系

2.什么是线性表？它的最主要的性质是什么？

2.1.1 线性表的定义

线性表(linear list)是具有相同特性的数据元素的一个有限序列。该序

Def

线性表是具有相同特性的数据元素的一个有限序列

最直观的就是数组，一个线性表 或者一种数据结构 应该具备增删改查的功能

特性如下

(1) 有穷性：一个线性表中的元素个数是有限的。

(2) 一致性：一个线性表中所有元素的性质相同。从实现的角度看,所有元素具有相同的数据类型。

(3) 序列性：一个线性表中所有元素之间的相对位置是线性的,即存在唯一的开始元素和终端元素,除此之外,每个元素只有唯一的前驱元素和后继元素。各元素在线性表中的位置只取决于它们的序号,所以在 一个线性表中可以存在两个值相同的元素。

3.什么是逻辑结构？什么是数据元素？什么是数据项？什么是数据类型？它们有什么联系？

1.1.2 逻辑结构

数据的逻辑结构是从数据元素的逻辑关系上描述数据的,是指数据元素之间的逻辑关系的整体,通常是从求解问题中提炼出来的。数据逻辑结构与数据的存储无关,是独立于计算机的,因此数据的逻辑结构可以看作是从具体问题抽象出来的数学模型。

在现实世界中,数据元素的逻辑关系是多种多样的,但在数据结构中主要讨论数据运算之间的相邻关系或者邻接关系。

数据的逻辑结构是数据元素之间的逻辑关系的整体

数据(data)是描述客观事物的数和字符的集合。例如,人们在日常生活中使用的各种文字、数字和特定符号都是数据。从计算机的角度看,数据是所有能被输入到计算机中,且能被计算机处理的符号的集合,它是计算机操作的对象总称,也是计算机所处理信息的某种特定的符号表示形式(例如,200902 班学生数据就是该班全体学生记录的集合)。

人们通常以**数据元素(data element)**作为数据的基本单位(例如,200902 班中的每个学生记录都是一个数据元素)。在有些情况下,数据元素也称为元素、结点、顶点或者记录等。一个数据元素可以由若干个数据项组成。

数据项(data item)是具有独立含义的数据最小单位,也称为字段或域。例如,200902 班中的每个数据元素(即学生记录)是由学号、姓名、性别和班号等数据项组成的。

数据对象(data object)是指性质相同的数据元素的集合,它是数据的一个子集。在数据结构课程中讨论的数据通常指的是数据对象。

数据结构(data structure)是指所有数据元素以及数据元素之间的关系,可以看作是相互之间存在着某种特定关系的数据元素的集合,如图 1.1 所示。因此,我们可以把数据结构看成是带结构的数据元素的集合。

而数据元素又是数据的基本单位 以教材上的例子 一个班, 我们要研究所有同学的学号, 身高, 年龄; 那么数据元素就是某一位同学 他作为一个基本数据元素 数据项就是这位同学具有的学号 身高 年龄 而数据对象就是 所有同学身高的集合 ...

数据类型则是区分某种数据的指标

数据元素包含各种不同数据类型的数据项 (字段)

不同数据元素之间的逻辑结构与存储结构构成数据结构

4.什么是顺序存储结构？什么是链式存储结构？还有哪些其他存储结构？

1. 顺序存储结构

顺序存储结构(sequential storage structure)是采用一组连续的存储单元存放所有的数据元素,也就是说,所有数据元素在存储器中占有一整块存储空间,而且两个逻辑上相邻的元素在存储器中的存储位置也相邻。因此,数据元素之间的逻辑关系由存储单元地址间的关系隐含表示,即顺序存储结构将数据的逻辑结构直接映射到存储结构。

顺序存储结构的主要优点是存储效率高,因为分配给数据的存储单元全用于存放数据元素,元素之间的逻辑关系没有占用额外的存储空间;另外,在采用这种存储方法时可实现对元素的随机存取,即每个元素对应一个逻辑序号,由该序号可直接计算出对应元素的存储地址,从而获取元素值。顺序存储结构的主要缺点是不便于数据修改,对元素的插入或删除

顺序存储结构表明存储数据的单元在空间上是连续的
最直观的例子就是数组

2. 链式存储结构

在**链式存储结构**(linked storage structure)中,每个逻辑元素用一个内存结点存储,每个结点是单独分配的,所有的结点地址不一定是连续的,所以无须占用一整块存储空间。为了表示元素之间的逻辑关系,给每个结点附加指针域,用于存放相邻结点的存储地址,也就是通过指针域将所有结点链接起来,这就是链式存储结构名称的由来。

链式存储结构的主要优点是便于数据修改,在对元素进行插入或删除操作时仅需修改相应结点的指针域,不必移动结点。与顺序存储结构相比,链式存储结构的主要缺点是存储空间的利用率较低,因为分配给元素的存储单元有一部分被用来存储结点之间的逻辑关系;另外,由于逻辑上相邻的元素在存储空间中不一定相邻,所以不能对元素进行随机存取。

链式存储结构表明存储数据的单元在空间上不一定连续,但是彼此之间存在逻辑关联

还存在索引存储结构,与哈希(散列)存储结构

前者是给每个存储的数据打一个索引标签,从而建立索引表

而后者则是将分布空间更大的数据通过哈希函数映射到一个较小的索引表上

对于上述四种数据结构 我们都可以从增删改查四个方面分析其长处与短处
对于增、删而言 链式存储 索引、散列存储比顺序要高效 但是对于查找来说
顺序存储 索引 散列效率更高

5.什么是单链表？什么是双向链表？什么是循环链表？什么是静态链表？静态链表中结点里包含的指针实际上代表什么？

单链表 -- 每个结点除开数据之外只有一个指向后序结点的指针

双链表 -- 每个结点除开数据之外有两个指针 一个指向前继结点，一个指向后序结点

循环链表 -- 链表中最后一个结点的后序指针指向头结点

静态链表是用数组实现的，指针实际上代表的是数组中元素的下标索引

6.什么是栈？给定一个入栈序列，如何判断一个出栈序列是否可能？什么是链式栈？跟顺序栈相比，它有什么优缺点？

栈是只能在某一端进行插入和删除数据的线性表

只给定入栈序列，出栈序列不唯一

链式栈是使用链表实现的栈

优点 -- 长度不固定，空间利用率高

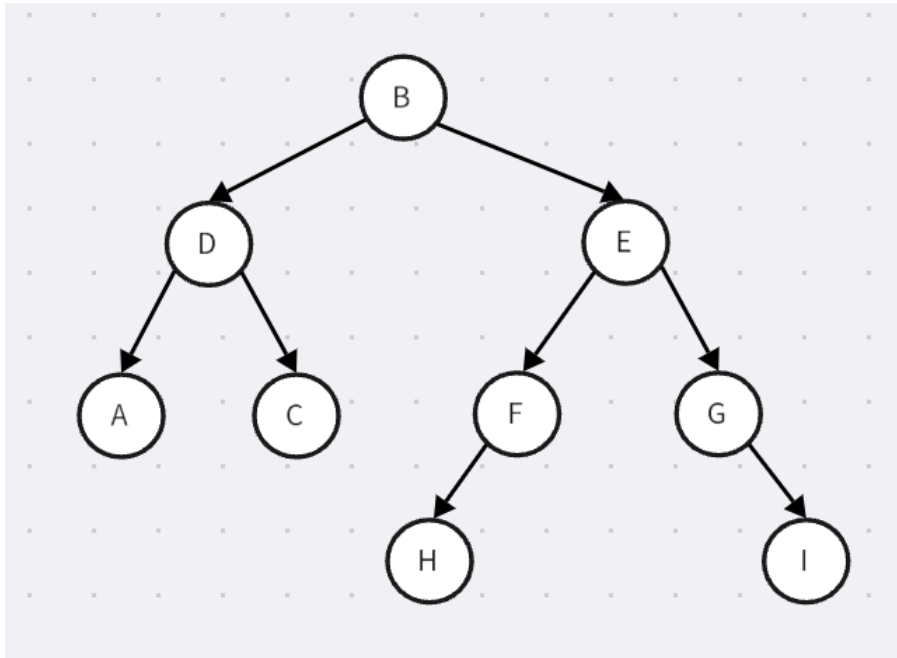
缺点 -- 访问效率比较低，需要进行遍历

7.怎样重构一棵二叉树？已知中序序列和后（先）序序列可以恢复一棵二叉树，怎样证明？已知中序序列和层次遍历序列，可以恢复二叉树吗？已知先序序列

和后序序列，是否也可以恢复二叉树呢？ 需要满足什么条件吗？ 如何证明？

重构二叉树，即根据某二叉树的前/中/后序遍历的结果，构造出原本的这颗二叉树

假设我们已知一颗二叉树 B



它的前序遍历结果 根 左 右

$BDACEFHGI$

也就是先打印当前结点，再往左子树访问，最后再往右子树访问

中序遍历结果 左 根 右

$ADCBHFEGI$

先访问左子树 再打印当前结点 最后访问右子树

后序遍历结果 左 右 根

$ACDHFIGEB$

还是按照左 右 根之间的关系依次遍历来求解比较简洁

那么假设我们知道了前序和中序遍历的结果

我们都能直接得到根结点就是 B

那么可以得到 左子树前序遍历的结果是 根左右为

$$DAC$$

左子树中序遍历的结果是 左根右为

$$ADC$$

于是根结点为 D 由此得到左子树的构成，相同的方法分析右子树的构成即可

如果是层次遍历的序列，我们也就是从上到下，每层从左往右

也就是

$$BDEACFGHI$$

可知根据层次遍历我们能快速找到根结点 B 从而得到左右子树中序遍历的结果

也就是 $ADC HFEGI$

此时注意到下一层应该是有两个结点，为 DE ,由此我们获得了左右子树的根结点 DE

此时左子树已重构，对于右子树，下一层至少有3个结点，除开前两个已经确定的 AC 我们得到右子树的这一层应该是 FG ，下一层对应是 HI 重构完毕

如果已知先序和后序遍历结果

我们这里也能快速的找到根结点 B

这里不难找到先序的左子树是 DAC 右子树是 $EFHGI$ 后序的左子树是 ACD

右子树是 $HFIGE$ 再找到根结点即可实现重构

条件是不能有重复的结点

8.树与二叉树有哪些区别与联系？树的常见的存储结构有哪些？

树是一种数据结构 每个结点只有一个前驱结点，但可以有0个或者多个后继结点

而二叉树的话后继节点最多为2

二叉树是树的一种子集

存储结构

1. 双亲存储结构 -- 即对每个结点，存放它的双亲结点的索引，当然只能有一个，它基于使用数组存储的顺序树
2. 子链存储结构 -- 基于链表存储的链性树结构，对每个结点，打一个链表数组，数组中每个元素都是指向子代的链表
3. 孩子兄弟链存储结构 -- 一个指向结点的长子结点 一个指向结点的兄弟结点

9.什么是哈夫曼树？什么是哈夫曼编码？怎样构造哈夫曼树？

7.81 哈夫曼树概述

在许多应用中经常将树中的结点赋予一个有某种意义的数值，称此数值为该结点的权。从根结点到该结点之间的路径长度与该结点上权的乘积称为结点的带权路径长度 (Weighted Path Length, WPL)。树中所有叶子结点的带权路径长度之和称为该树的带权路径长度，通常记为：

$$WPL = \sum_{i=1}^{n_0} w_i l_i$$

其中， n_0 表示叶子结点的个数， w_i 和 l_i 分别表示第 i 个叶子结点的权值和根到它之间的路径长度(即从根结点到该叶子结点的路径上经过的分支数)。

在 n_0 个带权叶子结点构成的所有二叉树中，带权路径长度 WPL 最小的二叉树称为哈夫曼树(Huffman tree)或最优二叉树。因为构造这种树的算法最早是由哈夫曼于 1952 年提出的，所以用他的名字命名。

扫一扫



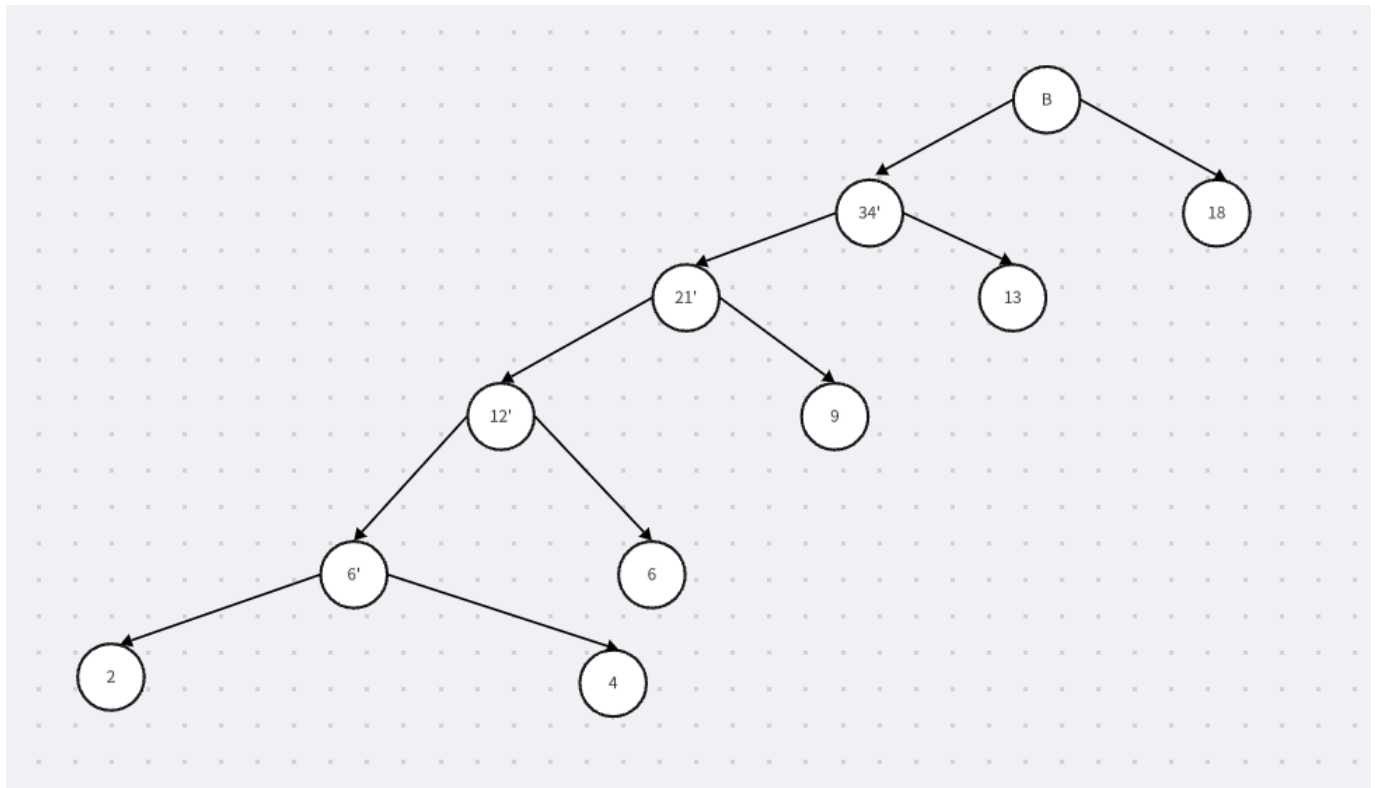
视频讲解

在一个只有叶子结点带权的二叉树中 我们计算带权路径长

$$WPL = \sum_{i=1}^{n_0} w_i l_i$$

哈夫曼树就是 WPL 最小的二叉树

构造哈夫曼树主要使用贪心算法，如下



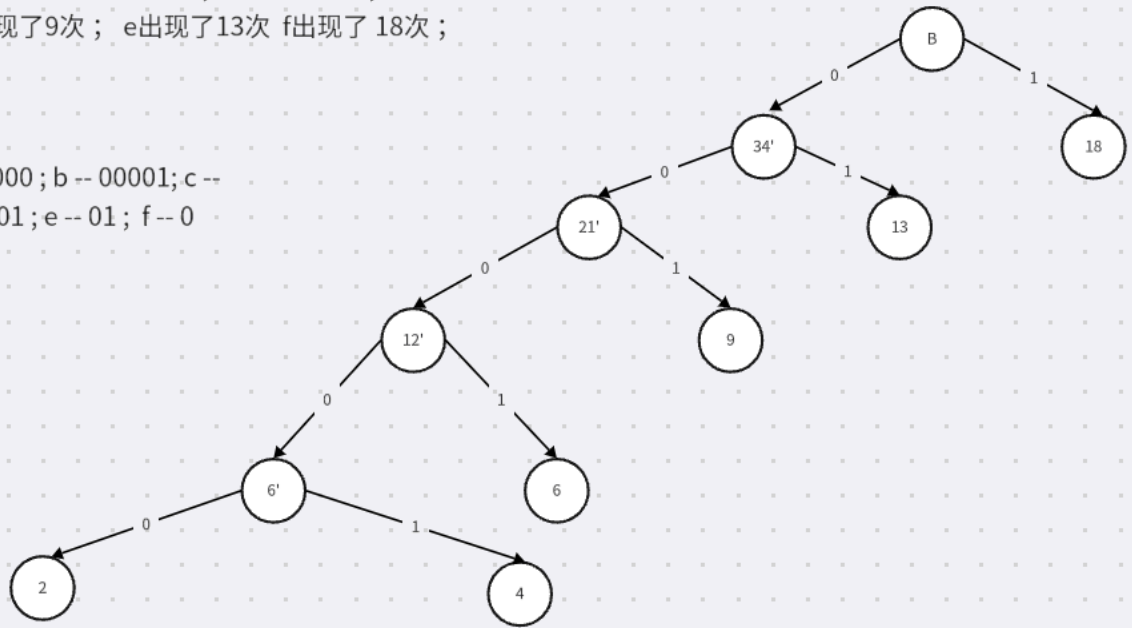
每次构建的时候，都选取最小的两个结点，作为叶子，逐层向上，使得最大的结点最靠近根结点，实现构造

哈夫曼编码则是根据字符在报文中出现的频率，以这个为大小判断依据构建最小哈夫曼树

然后以左边为 0 右边为 1 构建哈夫曼编码 如下

一段密文中 a出现了2次；b出现了4次；c出现了6次；
d出现了9次；e出现了13次 f出现了18次；

那么 a-- 00000；b-- 00001；c--
0001；d-- 001；e-- 01；f-- 0



越常用的字母我们用来编码它的信号就越少，节省了开销

10.完全二叉树的结点个数、叶子结点数和高度之间满足怎样的关系？如何给完全二叉树的结点编号？

首先，什么是完全二叉树？

若二叉树中最多只有最下面两层的结点的度数可以小于2，并且最下面一层的叶子结点都依次排列在该层最左边的位置上，则这样的二叉树称为**完全二叉树** (complete binary tree)，图 7. 5(b)所示为一棵完全二叉树。同样可以对完全二叉树中的每个结点进行层序编号，编号的方法和满二叉树相同，图 7. 5(b)中每个结点外边的数字为对该结点的编号。

最多只有下面两层能有叶子结点，其他的结点出度都是2 而且叶子结点都在最左边

在一棵二叉树中，如果所有分支结点都有左孩子结点和右孩子结点，并且叶子结点都集中在二叉树的最下一层，这样的二叉树称为**满二叉树** (full binary tree)。图 7. 5(a)所示就是一棵满二叉树。用户可以对满二叉树的结点进行**层序编号** (level coding)，约定编号从树

根为 1 开始,按照层数从小到大、同一层从左到右的次序进行,图 7.5(a)中每个结点外边的数字为对该结点的编号。当然也可以从结点个数和树高度之间的关系来定义,即一棵高度为 h 且有 2^h-1 个结点的二叉树称为满二叉树。

同理我们如此定义满二叉树 可知满二叉树是完全二叉树的一个特例
可知完全二叉树的结点个数最多的时候就是满二叉树, 结点最少的时候, 最后一层只有一个结点
即

$$2^{h-1} \leq n \leq 2^h - 1$$

同样的边界情况, 我们能得到叶子结点的数量关系

$$2^{h-2} \leq l \leq 2^{h-1}$$

我们按照层次遍历的思路来编号

11.什么是无向图？什么是有向图？什么是DFS？什么是BFS？进行DFS遍历和BFS遍历需要使用怎么的辅助数据结构？

8.1.1 图的定义

无论多么复杂的图都是由顶点和边构成的。采用形式化的定义，图(graph) G 由两个集合 V (vertex)和 E (edge)组成，记为 $G=(V,E)$ ，其中 V 是顶点的有限集合，记为 $V(G)$ ， E 是连接 V 中两个不同顶点(顶点对)的边的有限集合，记为 $E(G)$ 。

可以用字母或自然数来标识图中的顶点，这里约定用 $i(0 \leq i \leq n-1)$ 表示第 i 个顶点的编号，其中 n 为图中顶点的个数。当 $E(G)$ 为空集时，则图 G 只有顶点，没有边。

在图 G 中，如果表示边的顶点对(或序偶)是有序的，则称 G 为有向图(digraph)。在有向图中代表边的顶点对用尖括号括起来，用于表示一条有向边，如 $\langle i,j \rangle$ 表示从顶点 i 到顶点 j 的一条边，可见 $\langle i,j \rangle$ 和 $\langle j,i \rangle$ 是两条不同的边。

如果在图 G 中，若 $\langle i,j \rangle \in E(G)$ 必有 $\langle j,i \rangle \in E(G)$ ，即 $E(G)$ 是对称的，则用 (i,j) 代替这两个顶点对，表示顶点 i 与顶点 j 的一条无向边，则称 G 为无向图(undirgraph)。显然在无向图中 (i,j) 和 (j,i) 所代表的是同一条边。所以，无向图可以看成是有向图的特例。

图 8.1(a)所示为一个无向图 G_1 ，其顶点集合 $V(G_1)=\{0,1,2,3,4\}$ ，边集合 $E(G_1)=\{(1,2),(1,3),(1,0),(2,3),(3,0),(2,4),(3,4),(4,0)\}$ 。

图 8.1(b)所示为一个有向图 G_2 ，其顶点集合 $V(G_2)=\{0,1,2,3,4\}$ ，边集合 $E(G_2)=\{\langle 1,2 \rangle,\langle 1,3 \rangle,\langle 0,1 \rangle,\langle 2,3 \rangle,\langle 0,3 \rangle,\langle 2,4 \rangle,\langle 4,3 \rangle,\langle 4,0 \rangle\}$ 。

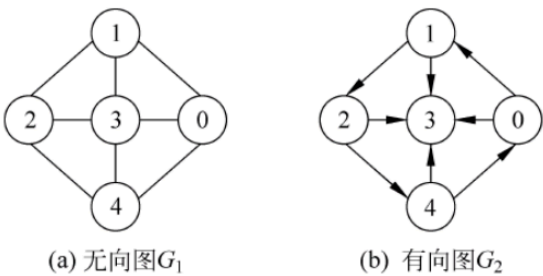


图 8.1 无向图 G_1 和有向图 G_2



图中每个结点的出结点和入结点都可以有多个，我们可以用 $\langle i,j \rangle$ 来表示图的一条边，如果 ij 的顺序不能颠倒，就说明结点之间的路径是单向的，也就是有向图

下面进行DFS，我们考虑使用邻接表的存储结构，即

822 邻接表存储方法

图的**邻接表**(adjacency list)是一种顺序与链式存储相结合的存储方法。

对于含有 n 个顶点的图,每个顶点建立一个单链表,第 $i(0 \leq i \leq n-1)$ 个单链表中的结点表示关联于顶点 i 的边(对有向图是以顶点 i 为起点的边),也就是将顶点 i 的所有邻接点(对有向图是出边邻接点)链接起来,其中每个结点表示一条边的信息。

每个单链表再附设一个头结点,并将所有头结点构成一个头结点数组 `adjlist`,`adjlist[i]` 表示顶点 i 的单链表的头结点,这样就可以通过顶点 i 快速找到对应的单链表。

在邻接表中有两种类型的结点,一种是头结点,其个数恰好为图中顶点的个数;另一种是边结点,也就是单链表中的结点。对于无向图,这类结点的个数等于边数的两倍;对于有向图,这类结点的个数等于边数。

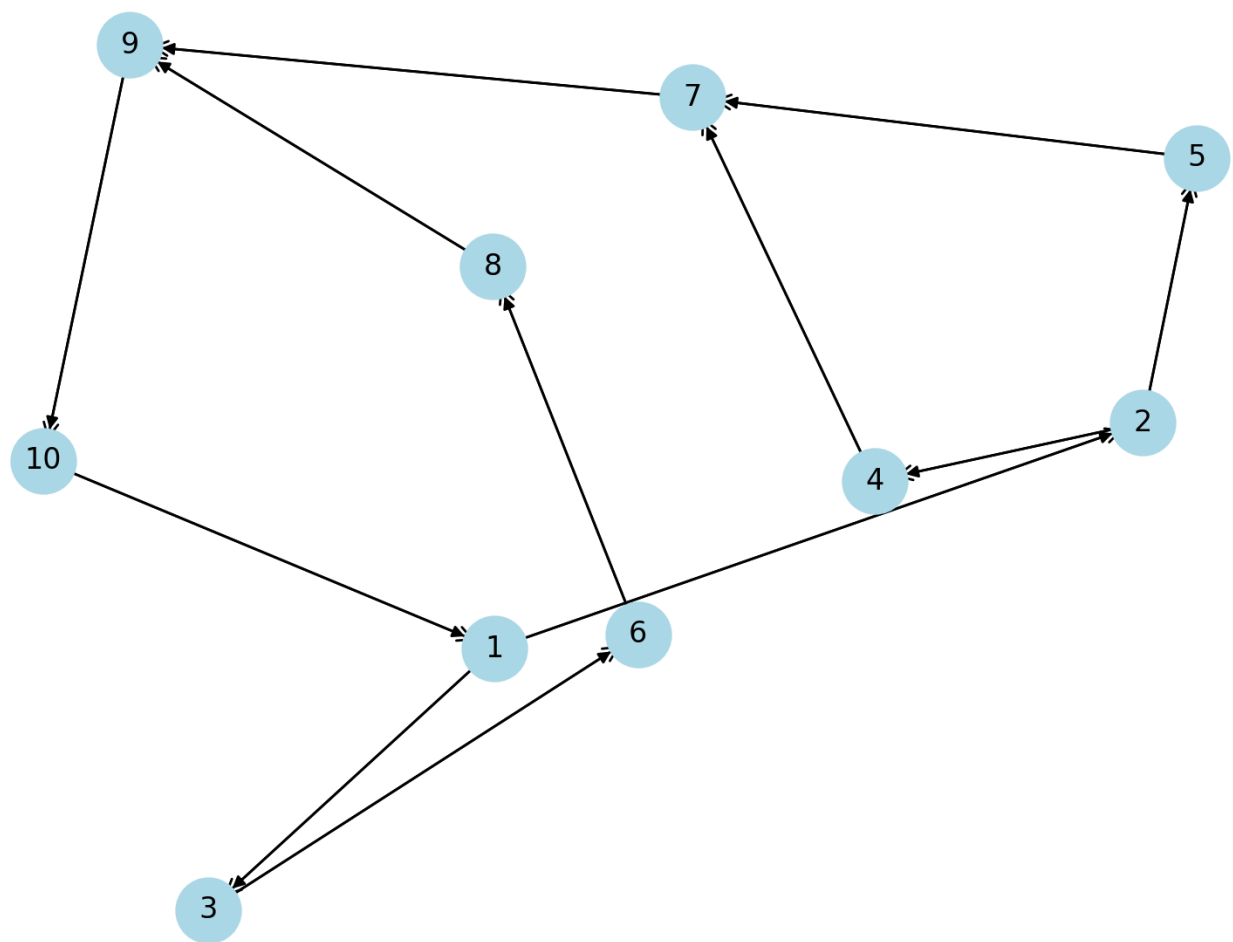
边结点和头结点的结构如下:

边 结 点			头 结 点	
adjvex	nextarc	weight	data	firstarc

其中,边结点由 3 个域组成,`adjvex` 表示与顶点 i 邻接的顶点编号,`nextarc` 指向下一个边结点,`weight` 存储与该边相关的信息,如权值等。头结点由两个域组成,`data` 存储顶点 i 的名称或其他信息,`firstarc` 指向顶点 i 的单链表中的首结点。



对每个结点, 都连接上一个链表, 链表中每个结点都指向当前结点能通向的后继结点



DFS 即对于一个结点，优先往深处访问没有访问过的结点，即往前走一条路

譬如 我们从1开始

1 -- 2 -- 4 -- 7 -- 9 -- 10 -- 1,已经访问，我们回溯到2 即

1 -- 2 -- 4 -- 7 -- 9 -- 10 ----回溯到2 -- 5 -- 7 (已访问)

1 -- 2 -- 4 -- 7 -- 9 -- 10 ----回溯到2 -- 5回溯到 1 -- 3 -- 6 -- 8 -- 9

(访问完毕)

最终序列为 1 -- 2 -- 4 -- 7 -- 9 -- 10 -- 5 -- 3 -- 6 -- 8

走不下去，就回溯到能走为止

BFS 对于一个结点 我们优先把所有能走的路线都给走掉 再按照下标从小到大的顺序 对每个结点重复上述过程

从1开始

能走的有2 3 故序列当前为 1 2 3

再从2开始 4 和 5 都能走 当前序列为 1 2 3 4 5 再对于3 只有 6 能走 序列为 1 2 3 4 5 6

那么这一层就是 4 5 6 从 4开始 只能走 7 5也是只能走 7 6能走 8 当前序列 1 2 3 4 5 6 7 8, 下一层是 7 8 下下层为 9 最后一层为 10

序列为

1 2 3 4 5 6 7 8 9 10

DFS使用的是回溯的思想, 那么自然而然就使用栈 递归的写法是用函数调用的递归关系, 不让写递归就手搓栈了

BFS则是对每个结点都要访问它的后继, 那么就是队列了, 在当前结点出队列的时候, 子节点全部进队列

12.什么是B-树? 怎样构造B-树? 什么是B+树? B-树与B+树的差别在哪里?

B-树(B tree)中所有结点的孩子结点的最大值称为 B-树的阶,通常用 m 表示,从查找效率考虑,要求 $m \geq 3$ 。一棵 m 阶 B-树或者是一棵空树,或者是满足下列要求的 m 叉树:

- (1) 树中每个结点最多有 m 棵子树(即最多含有 $m-1$ 个关键字,设 $\text{Max}=m-1$);
- (2) 若根结点不是叶子结点,则根结点最少有两棵子树;
- (3) 除根结点以外,所有非叶子结点最少有 $\lceil m/2 \rceil$ 棵子树(即最少含有 $\lceil m/2 \rceil - 1$ 个关键字,设 $\text{Min}=\lceil m/2 \rceil - 1$);
- (4) 每个结点的结构为:

n	p_0	k_1	p_1	k_2	p_2	$\dots$	k_n	p_n
-----	-------	-------	-------	-------	-------	---------	-------	-------

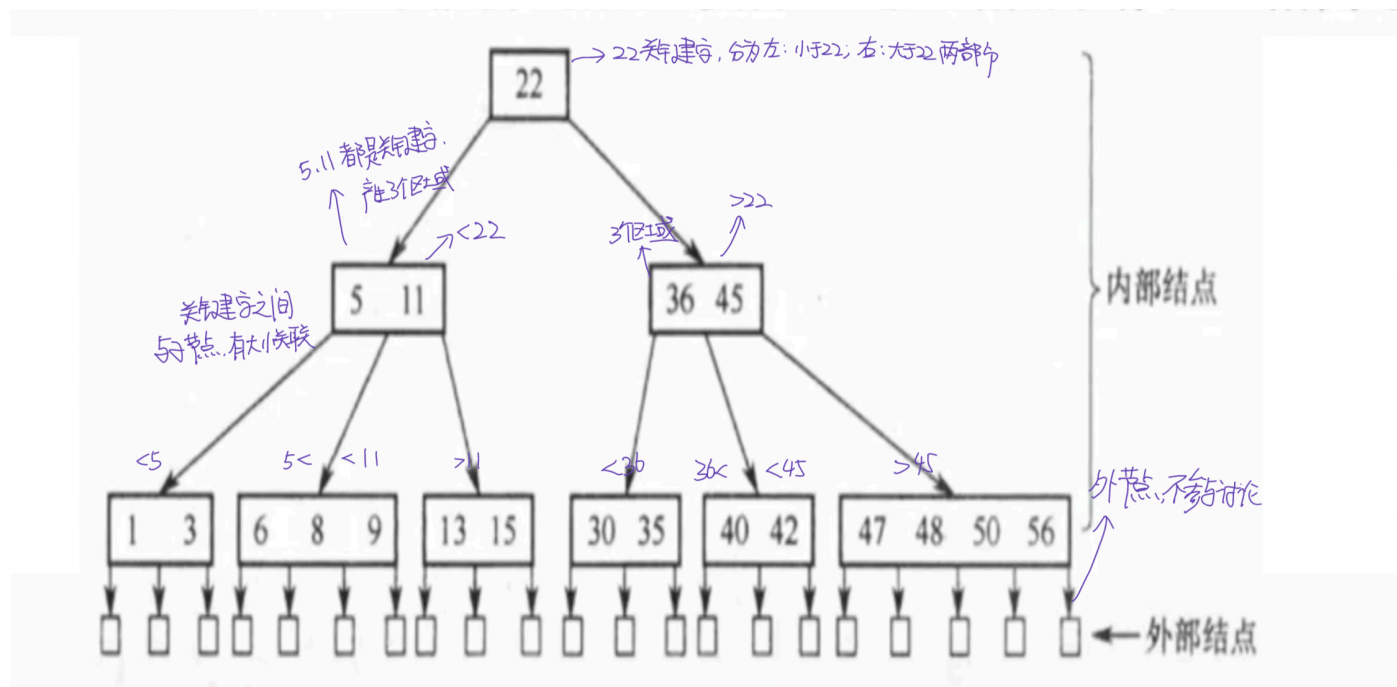
其中, n 为该结点中的关键字个数,除根结点以外,其他所有结点的关键字个数 n 满足 $\lceil m/2 \rceil - 1 \leq n \leq m-1$; $k_i (1 \leq i \leq n)$ 为该结点的关键字且满足 $k_i < k_{i+1}$; $p_i (0 \leq i \leq n)$ 为该结点的孩子结点指针,满足 $p_i (0 \leq i \leq n-1)$ 所指子树上结点的关键字均大于 k_i 且小于 k_{i+1} ; p_n 所指子树上结点的关键字均大于 k_n 。

- (5) 所有的外部结点在同一层,并且不带信息。

我们基于处理题目的角度来分析一下B树

首先明确两个概念，B树的阶 结点 关键字

我们认为 B树的关键字是按照一个确定的大小关系分布的，高处结点会被里面的关键字划分为几个部分，这几个部分往下指向的结点内部的关键字应该和高处的关键字建立一定的联系，看下面的例子



明白关键字和结点的关系，我们再考虑阶的问题

阶 m 保证了B树结点能有子节点的个数,与内部关键字的个数

对于 m 阶B树,它的一个结点中的关键字个数 n 满足

$$\lceil \frac{m}{2} \rceil + 1 \leq n \leq m - 1$$

一个结点最多有 m 个子结点

接下来是B树的构建,我们以一道408来分析一下

10. 依次将关键字 5, 6, 9, 13, 8, 2, 12, 15 插入初始为空的 4 阶 B 树后, 根节点中包含的关键字是:

- A、8 B、6, 9 C、8, 13 D、9, 12

按顺序, 从左往右, 从下往上.

先确定关键字个数, $m=4 \Rightarrow \lceil \frac{4}{2} \rceil - 1 \leq n \leq m-1 \Rightarrow 1 \leq n \leq 3$.

先: 5, 6, 9, 13 $\Rightarrow$ 超过这个把前三个中间元素向上, 做为父节点关键字

$\Rightarrow$ 5 9, 13, 把 6 提 8, 即 5 8 9 13, 再是 2 $\Rightarrow$ 2 5 8 9 13, 再 12, 把 8, 9, 13 中间提来

2, 5 8 12 13, 再是 15 $\Rightarrow$ 2, 5 8 12 13 15 符合要求

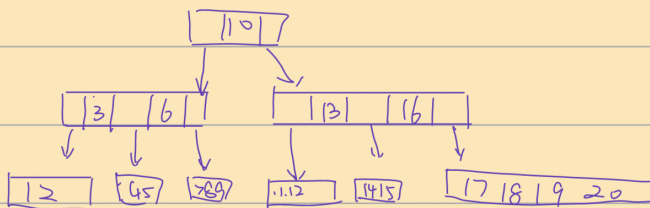
总之 B 树的关键在于确定关键字的数量关系 也就是阶 然后合理的排列关键字

再给一个例子

5 阶 B 树, 关键字为 1, 2, 6, 7, 11, 4, 8, 13, 10, 5, 17, 9, 16, 20, 3, 12, 14, 18, 19, 15

$\Rightarrow$ 个数要求 $\lceil \frac{5}{2} \rceil - 1 \leq n \leq 5-1 \Rightarrow 2 \leq n \leq 4$. 从左到右, 从下到上.

① 1, 2, 6, 7 $\leftarrow$ 11, 抬 6 $\Rightarrow$ 12 $\leftarrow$ 7, 11
 ② 12, 4 $\leftarrow$ 7, 8, 11, 13 $\leftarrow$ 10, 抬 11 $\Rightarrow$ 12, 4 $\leftarrow$ 7, 8 $\leftarrow$ 10, 13 $\rightarrow$ 交换成 12, 4 $\leftarrow$ 7, 8 $\leftarrow$ 10, 13
 ③ 12, 4, 5 $\leftarrow$ 7, 8, 9 $\leftarrow$ 10, 11, 13, 17 $\leftarrow$ 16, 抬 16 $\Rightarrow$ 12, 4, 5 $\leftarrow$ 7, 8, 9 $\leftarrow$ 10, 11, 13, 17
 ④ 12, 4, 5 $\leftarrow$ 7, 8, 9 $\leftarrow$ 10, 11, 13, 17, 20 $\leftarrow$ 16, 3, 12, 14, 18, 19, 20 $\leftarrow$ 16, 抬 16 $\Rightarrow$ 12, 4, 5 $\leftarrow$ 7, 8, 9 $\leftarrow$ 10, 11, 13, 17, 20
 ⑤ 12, 4, 5 $\leftarrow$ 7, 8, 9 $\leftarrow$ 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20 $\leftarrow$ 16, 抬 16 $\Rightarrow$ 12, 4, 5 $\leftarrow$ 7, 8, 9 $\leftarrow$ 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20



从左到右读 顺序要递增

删除的时候也是类似的思路,关键是满足阶以及关键字顺序的要求

9.34 B+ 树

在索引文件组织中经常使用 B₋ 树的一些变形,其中 B₊ 树(B₊ tree)是一种应用广泛的变形。一棵 m 阶 B₊ 树满足下列条件:

- (1) 每个分支结点最多有 m 棵子树。
- (2) 根结点或者没有子树,或者最少有两棵子树。
- (3) 除根结点以外,其他每个分支结点最少有 $\lceil m/2 \rceil$ 棵子树。
- (4) 有 n 棵子树的结点有 n 个关键字。

(5) 所有叶子结点包含全部关键字及指向相应记录的指针,而且叶子结点按关键字大小顺序链接(可以把每个叶子结点看成是一个基本索引块,它的指针不再指向另一级索引块,而是直接指向数据文件中的记录)。

(6) 所有分支结点(可看成是索引的索引)中仅包含它的各个子结点(即下级索引的索引块)中的最大关键字及指向子结点的指针。

例如,图 9.28 所示为一棵 4 阶的 B₊ 树,其中叶子结点的每个关键字下面的指针表示指向对应记录的存储位置。通常在 B₊ 树上有两个头指针,一个指向根结点,这里为 root,另一个指向关键字最小的叶子结点,这里为 sqt。

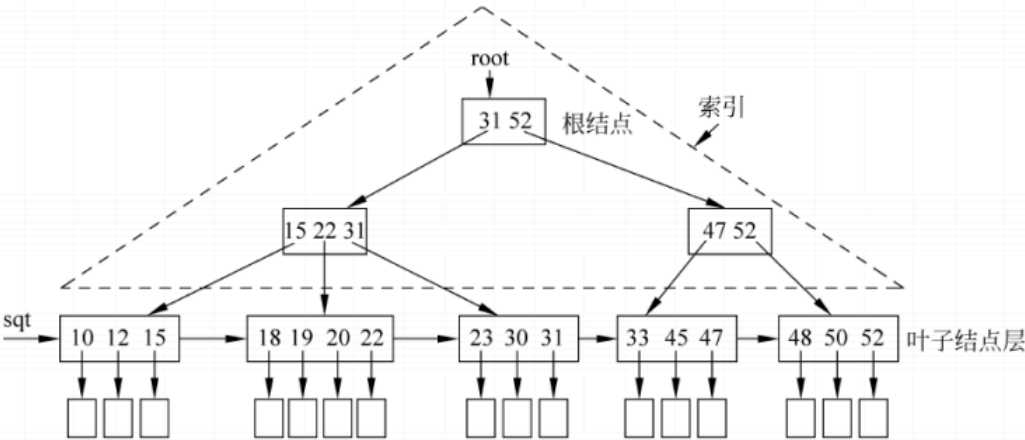


图 9.28 一棵 4 阶的 B₊ 树

B+树

13.什么是递归？如何在二叉树上利用递归完成基于深度优先策略的遍历？如何在遍历过程中完成特点任务的计算（如计算路径长度）？

递归也就是函数在执行过程中调用自身的过程

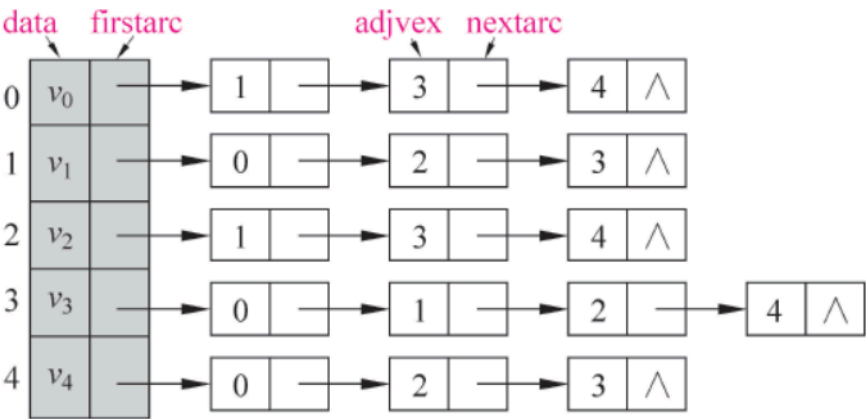
如果要使用递归实现DFS 即使用邻接表,若当前的结点还没有访问,就调用自

身,去搜索邻接表中的下一个结点

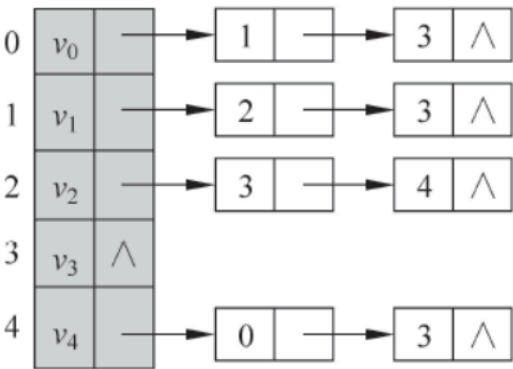
```
int visited[MAX] = {0}; //全局数组
void DFS(AdjGraph * G, int v) //深度优先遍历算法
{   ArcNode * p;
    visited[v] = 1; //置已访问标记
    printf("%d ", v); //输出被访问顶点的编号
    p = G->adjlist[v].firstarc; //p 指向顶点 v 的第一个邻接点
    while (p != NULL)
    {   if (visited[p->adjvex] == 0) //若 p->adjvex 顶点未被访问,递归访问它
        DFS(G, p->adjvex);
        p = p->nextarc; //p 指向顶点 v 的下一个邻接点
    }
}
```

如果要计算路径长度的话可以在递归的时候同时维护一个路径变量,每进一层就自增,然后在递归终结的时候输出路径长度,或者使用栈

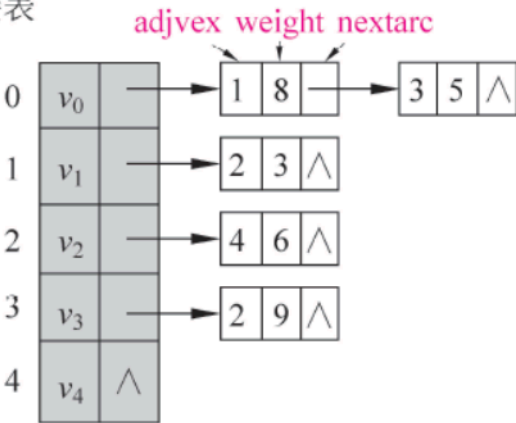
14.什么是图的邻接表、逆邻接表、十字链表、邻接表多重表？



(a) 图 G_1 的邻接表



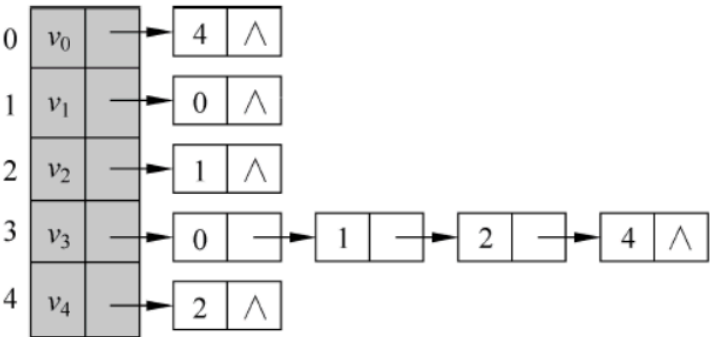
(b) 图 G_2 的邻接表



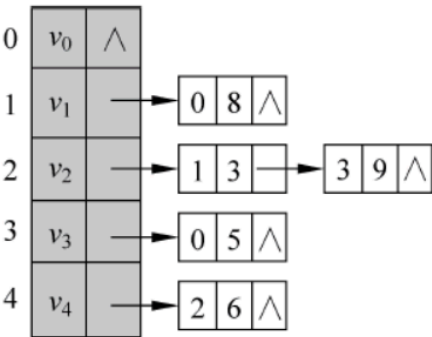
(c) 图 G_3 的邻接表

图 8.7 3 个邻接表

邻接表如下 表示一个结点它的后继结点有哪些

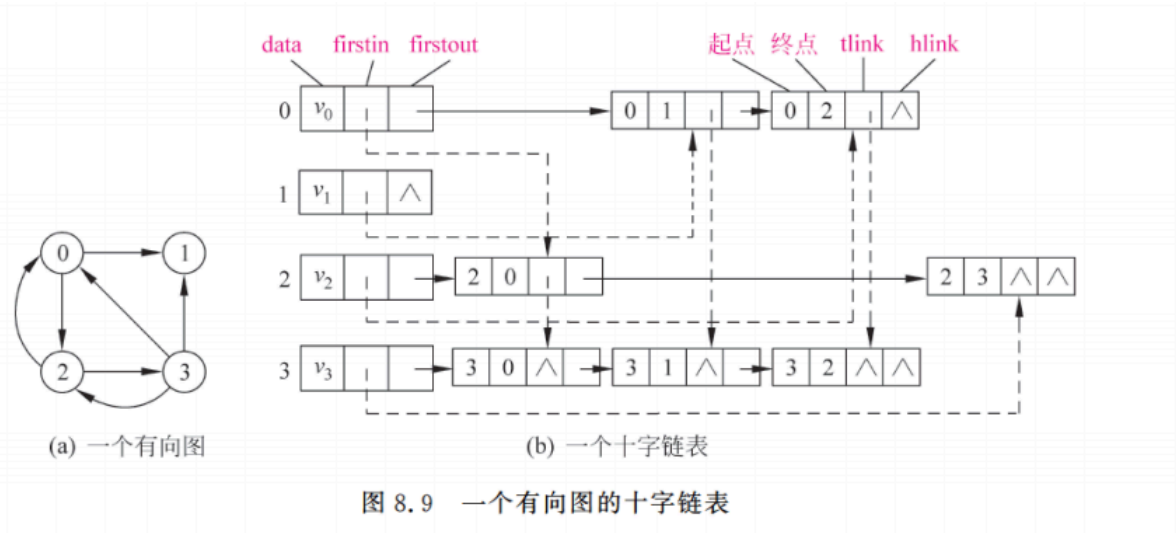


(a) 图 G_2 的逆邻接表



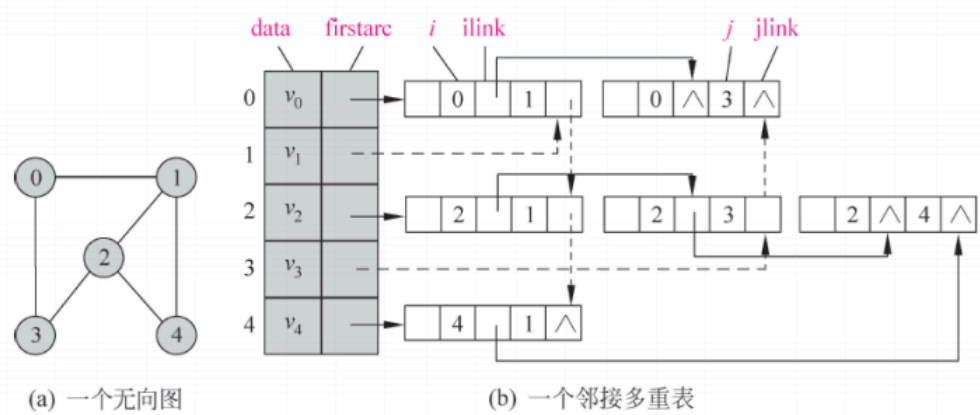
(b) 图 G_3 的逆邻接表

逆邻接表则表示一个结点它的前驱结点有哪些



2. 邻接多重表

邻接多重表(adjacency multi-list)是无向图的另外一种存储结构,与十字链表类似。下面以图 8.10(a)所示的无向图来说明邻接多重表的创建过程。



15.什么是循环队列？ rear和front指针通常代表什么？ 改变rear和front的定义之后， 如何进行相关操作？

循环队列表示队列的外表呈现一个头尾相连的圆形, **rear**表示尾指针,也就是队列的下一个元素的位置,标记队列中尾元素的索引

front表示头指针,标记队列中第一个元素的索引

判断环形队列为空//满的依据是

$$rear \equiv front \bmod n$$

此时为空

$$rear + 1 \equiv front \bmod n$$

此时为满

如果修改定义,我们注意这两个式子

16.什么是分块查找? 如何进行分块查找? 如何计算分块查找的效率?

对于数据块,我们可以进行顺序查找,复杂度 $O(N)$, ASL为 $\frac{N+1}{2}$ 如果已知有序,可以进行折半查找,复杂度为 $O(\log N)$, ASL为 $\log_2(N+1) - 1$

分块查找则是把 N 条数据分为 m 个块,每个块满足上下界是递增分布的,那么先对块用折半查找,这一步的ASL为 $\log_2(m+1) - 1$ 接下来对这个块,长度为 $\frac{N}{m}$ 进行顺序查找,ASL为 $\frac{\frac{N}{m}+1}{2}$,那么总

$$ASL = \frac{\frac{N}{m}}{2} + \log_2(m+1)$$

这是用折半查找寻找块的ASL

如果是顺序查找来找到块,那么

$$ASL = \frac{m+1}{2} + \frac{\frac{N}{m}+1}{2}$$

此时就能计算当

$$m = \sqrt{N}$$

的时候,ASL最小

17.什么是二叉排序树？什么是AVL树？它们有什么性质？

二叉排序树--**BST** 满足对于任何子树,都有左<中<右的关系,来几个例子

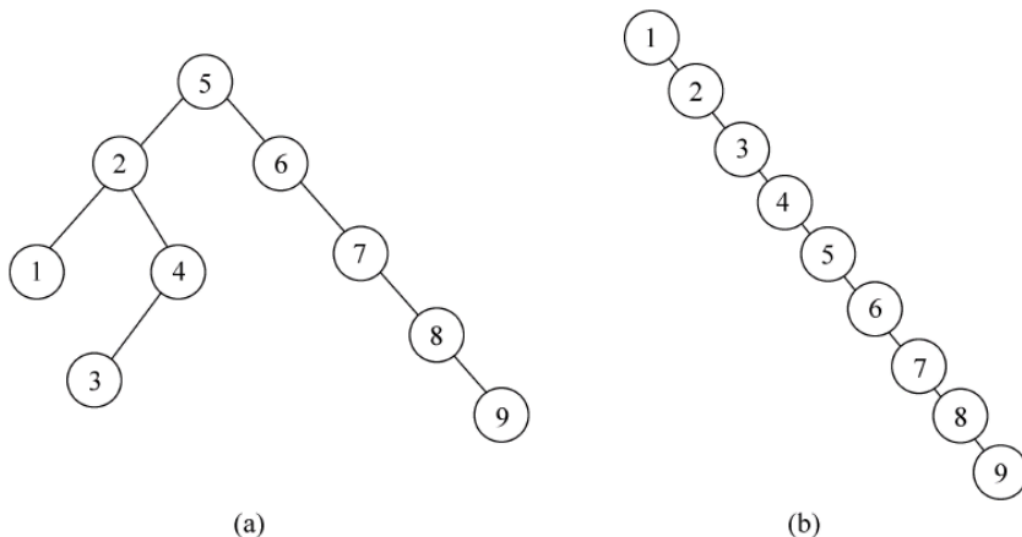


图 9.10 两棵二叉排序树

我们先分析搜索它的成功与失败的ASL

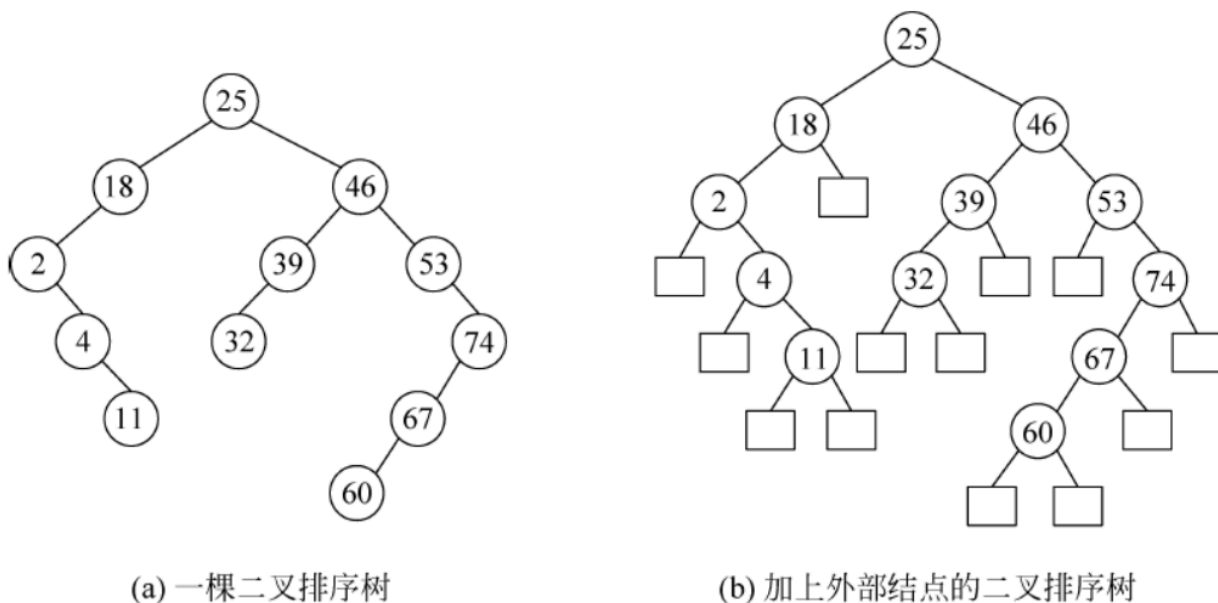


图 9.11 一棵二叉排序树及加上外部结点以后

如果搜索成功 --- 所搜结点在二叉树中,即左图

$$ASL_{success} = \frac{1 * 1 + 2 * 2 + 3 * 3 + 4 * 3 + 5 * 2 + 6 * 1}{1 + 2 + 3 + 3 + 2 + 1}$$

如果搜索失败 --- 所搜结点不在二叉树中,考虑右图,也就是全找到了外部节点去了

$$ASL_{defeat} = \frac{2 * 1 + 3 * 3 + 4 * 4 + 5 * 3 + 6 * 2}{1 + 3 + 4 + 3 + 2}$$

AVL,即平衡二叉树,我们先定义

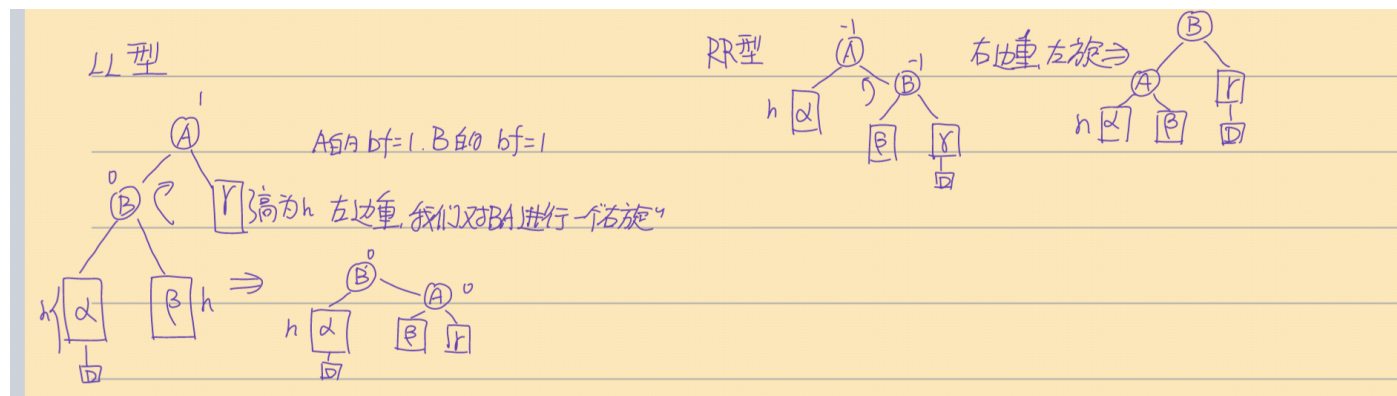
① 平衡因子 bf

$bf = h_l - h_r$ 也就是左子树的高度减去右子树的高度

AVL树中每个结点的 bf 满足 $|bf| \leq 1$

然后讲一下怎么来配平一个BST

分为LL,RR,LR,RL型,我们主要分析LL和,RR



并查集结构的时候主要是对每一个元素都建立起它的父元素数组 `parent` 我们举个例子

假设有5个元素 $1, \dots, 5$ 他们一开始的父元素数组长这样 `parent = {1, 2, 3, 4, 5}` 我们对 `1, 2` 执行一次 `Union` 对 `4, 5` 执行一次 `Union` 最终得到的 `parent = {1, 1, 3, 4, 4}`, 然后我们再对 `5` 进行一次 `Find`, 返回的是它的父亲, 也就是 `4`

在使用 `Kruskal` 算法求图的最小生成树的时候, 我们每次都得判断当前图中最小边 e_{min} 的两侧结点是否属于不同的集合 U 、 V 也就是执行一次 `Find` 操作, 再考虑是否合并

综上, 构建并查集的时间复杂度为 $O(N)$

19. 什么是图的单源最短路径问题? 解决该问题的常见方法有哪些?

也就是选定图中的一个点, 求它到图中其他的不同的点的最短距离, 在正权图中, 主要考虑 `dijkstra` 算法, 这里简要描述一下它的过程

1. 选定一个图中的点 A , 用邻接矩阵标记其他点到这个点的距离, 没有直接连通的点距离标记为 ∞ , 同时定义一个数组 `mark` 用来标记是否找到最短路径
2. 遍历所有与 A 相邻的点, 找到路径最短的点 B , 此时我们就已经找到了 A 到 B 的最短路径, $mark[B] = 1$ 并记录长度
3. 然后遍历所有与 B 相邻的结点, 比较 $AB + BE_i$ 与 AE_i 之间的关系, 如果前者小, 就对路径数组进行更新
4. 然后我们再寻找当前路径距离 A 最近的结点 C , $mark[C] = 1$, 更新路径数组, 如下循环

20.什么是快速排序？在不同条件下的算法复杂度是什么？

快速排序，也就是选择一个基准 **pivot**，然后遍历当前数组，把所有小于 **pivot** 的放到它前面，大于 **pivot** 的放到它后面

然后再对前后两个小数组选择新的 **pivot** 进行递归化的排序

- 较优情况--每次选择的 **pivot** 大小比较居中

此时我们的问题都能被拆解为近似一半的部分，总共需要 $O(\log N)$ 的拆解步骤

第一次拆分，你需要遍历整个数组，

第二次拆分，你需要同时遍历前后两个小数组

....

所以每次你比较的复杂度都是 $O(N)$

总的时间复杂度也就是 $O(N \log N)$

- 最坏情况--每次选择的 **pivot** 都太大或者太小

这种情况下，我们每次选择的 **pivot** 往往只能帮我们减少几个问题，我们要拆分的次数就是 $O(N)$ 级别的，从这往下，总的时间复杂度就应该是 $O(N^2)$

平均复杂度还是 $O(N \log N)$

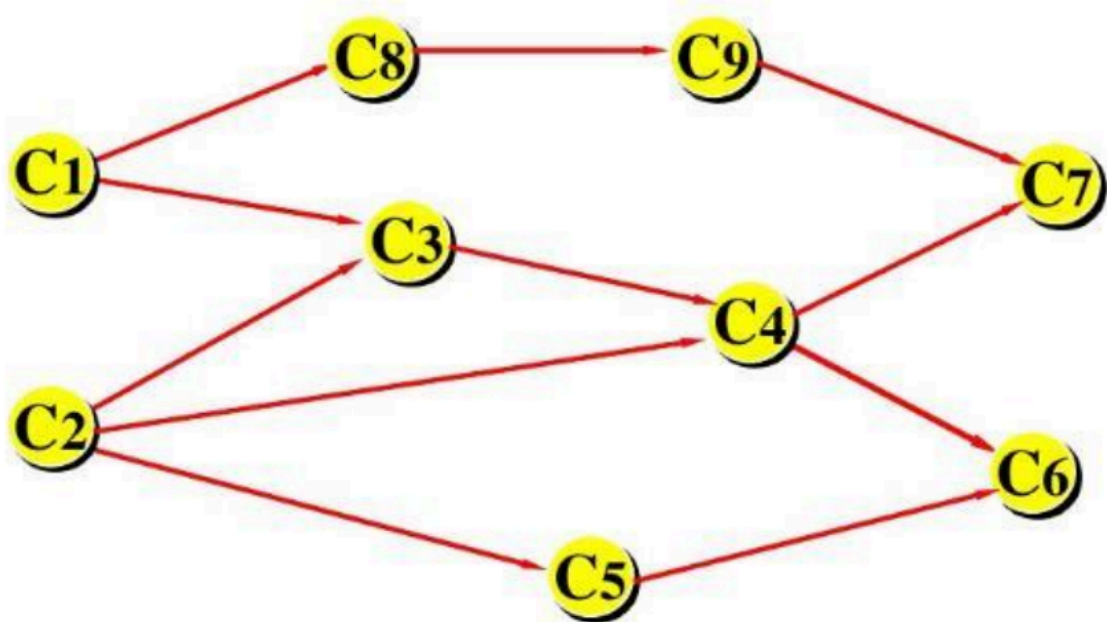
21. 什么是AOE网络？结点和边分别代表什么？什么关键活动？怎样求关键路径？

我们顺带连着AOV网一起讲了，

- AOV网

一个有向图，用顶点表示活动，顶点之间的连线表示了活动之间的先后

关系



这就是一个AOV网，可见 C_1, C_2 是 C_3 的前置条件...

我们主要的处理方式是拓扑排序，流程如下

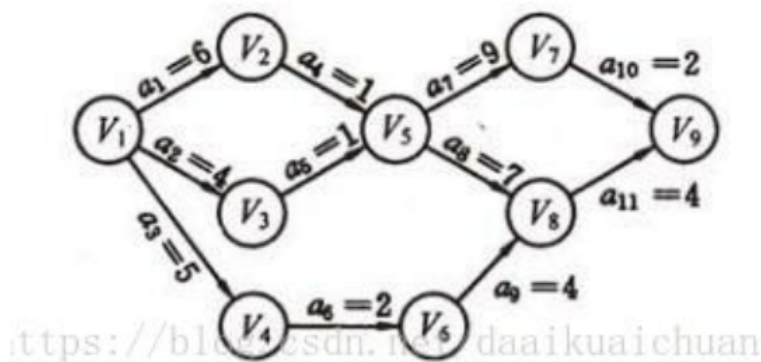
1. 选定图中入度为0的结点，输出它，并且删除它和它的所有出边，更新图中结点的入度
2. 重复以上流程，直到没有结点

我们按照从小到大的删除顺序，不难得到上述图的一个拓扑序列是

$C_1 C_2 C_3 C_4 C_5 C_6 C_8 C_9 C_7$

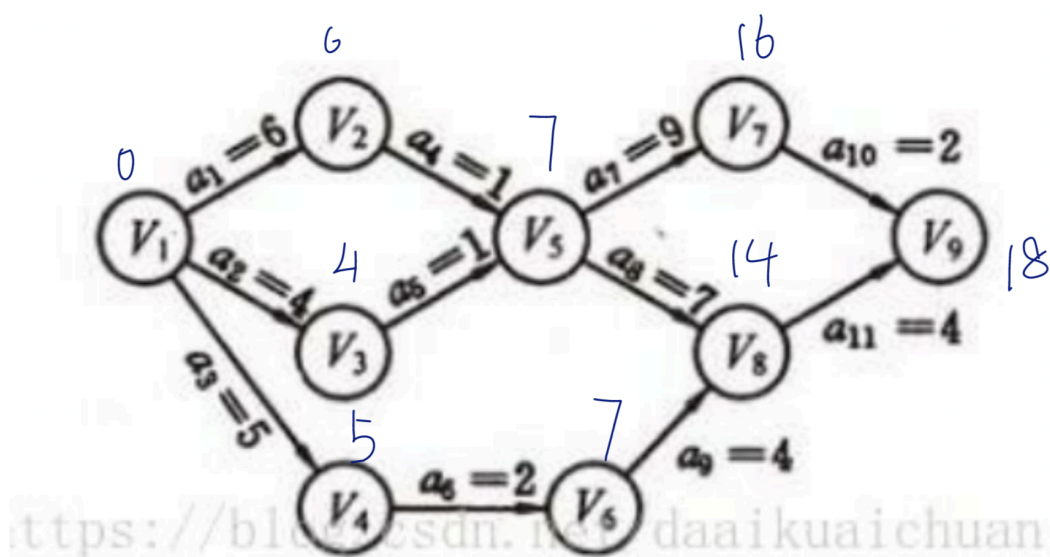
- AOE网

一个带权有向图，用结点表示事件的不同状态/步骤，用边表示完成事件的先后关系和所用时间，如下



可以把各个结点理解为一个工程，我们记**入度**为零的点为**起点**，**出度**为零的点为**汇点**，从起点到汇点，就是一项工程的完成，有的活动占用时间多，需要早点完成；有的活动占用时间少，可以稍微拖延
故在AOE网中，我们很重要的一个问题就是求解各时间的**最早开始时间**和**最晚开始时间**

同样可以采用拓扑排序的方法来求解前者

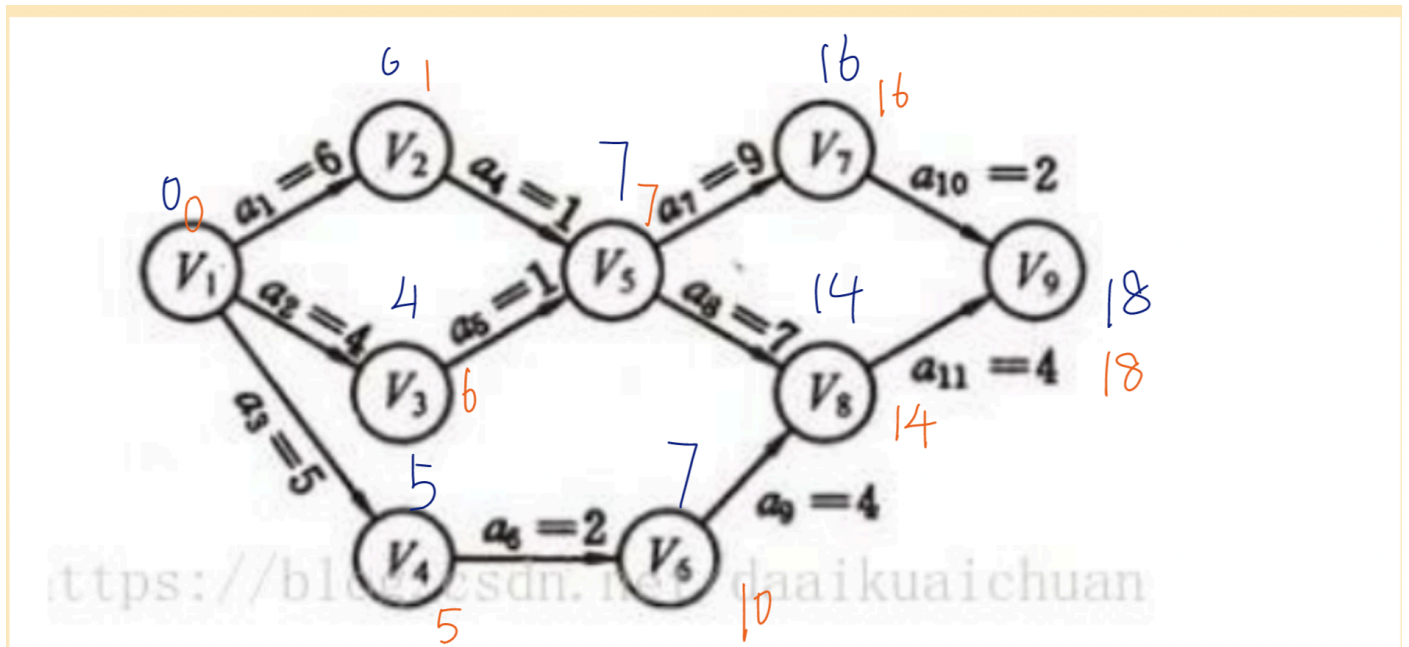


我们给所有结点的最早开始时间设为0，也就是直接上手开始做，然后按照边的权值，给他的子结点赋值，也就是我们直接从起点开始做的时候，也要这么久才能到这些子结点，
然后遍历每个子结点（此时入度都是0）如果到某个结点（如这里的 V_5

) 有多个时间, 我们取**大**的 (也就是**更耗时**的路径) 毕竟我们都要等哪些耗时的做完才能开始下一个工程

然后用逆拓扑排序的思想求解最晚开始时间,

我们在上图中已经求出来了, 我们最快也要18个单位时间才能到达 V_9 , 我们再把所有项目的最晚开始时间都设置的和 V_9 一样, 然后找到每个**出度**为0的点, 删去它和它的所有入边, 并挨个给结点的最晚开始时间减去权值



V_7 和 V_4 都能回到 V_5 ,但是时间都是一样的, 如果不一样我们反过来取小的, 因为如果取了大的, 这也就意味着你在正着去做的时候, 就一定会超时了 然后我们把最早开始时间等于最晚开始时间的结点标记为关键结点, 也就是拖都不能拖的结点, 从此构建出关键线路, 这里有两条

22. 什么是多关键字排序? 什么是基数排序算法? 算法复杂度如何?

多关键字排序, 也就是基于多个标准对一个实例进行排序, 在实际生活中比较常见

而基数排序则是根据一种有限而且易于分辨的基数指标，把所有的元素按照基数分类并且存储

譬如，已知有1000个5个字母长的字符串，就可以按照首字母为基数，分出26个不同的区域，进行一个序的排，很显然的空间换时间，空间复杂度

$O(m * n)$ ，时间复杂度 $O(m + n)$

，在这种每个排序元素的基数都不多的情况下

23. 什么是KMP算法？什么是目标串？什么是模式串？next数组起到什么作用？

KMP，也就是字符串匹配算法，目标串往往是很长的一个字符串，形如 *asasadadsadafdadsasafdasda* 我们需要在其中找到模式串 *dsada* 是否匹配 我们匹配的时候一般都是两个串的头对齐，然后指针挨个移动比较，但是如果出现的失配，在BF算法中，我们的指针就得移动回模式串的头，再把模式串的头与目标串的第二个字符对其，来匹配，而 *next* 数组则是根据匹配失败时我们的已知信息，来调整下次匹配开始的位置，我们已知以下事实

模式串失配前的所有部分都是和目标串匹配的

我们假设模式串为

ababceeeababd'eeeababd

假设我们在 *d'* 的位置失配了，但是我们知道前面的部分都是匹配的，于是我们观察模式串，最前面的四位 *abab* 和失配位置的前面四位 *abab* 都是匹配的，也就是说如果我们把模式串移动到失配位置的前4位，我们都能匹配上，这样就大大减少了运行开销，也就是 $next[12] = 3$ ，如果再不匹配，再找 *next* 来回退，如果 *next* 是 -1，就两个指针同时向右

接下来我们只要分析如何求出 $next$

伪代码如下

```
function computeNextArray(P):
    m = length of P
    next[0] = -1
    j = 0 // j 用来跟踪前缀的长度

    for i = 1 to m-1:
        while j > 0 and P[i] != P[j]:
            j = next[j-1] // 找到更短的匹配前缀

        if P[i] == P[j]:
            j += 1 // 如果匹配, 增加前缀长度
        next[i] = j // 更新 next 数组

    return next
```

KMP的时间复杂度是 $O(m + n)$

24. 常用的排序算法有哪些？哪些排序算法是稳定的，哪些是不稳定的？这些排序算法的时间复杂度和空间复杂度如何？单趟排序能决定某个（些）元素的最终位置的排序算法有哪些

前三问：

排序算法	平均时间复杂度	最好情况	最坏情况	空间复杂度	排序方式	稳定性
冒泡排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	In-place	不稳定
插入排序	$O(n^2)$	$O(n)$	$O(n^2)$	$O(1)$	In-place	稳定
希尔排序	$O(n \log n)$	$O(n \log^2 n)$	$O(n \log^2 n)$	$O(1)$	In-place	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Out-place	稳定
快速排序	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	In-place	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	In-place	不稳定
计数排序	$O(n + k)$	$O(n + k)$	$O(n + k)$	$O(k)$	Out-place	稳定
桶排序	$O(n + k)$	$O(n + k)$	$O(n^2)$	$O(n + k)$	Out-place	稳定
基数排序	$O(n \times k)$	$O(n \times k)$	$O(n \times k)$	$O(n + k)$	Out-place	稳定

第四问

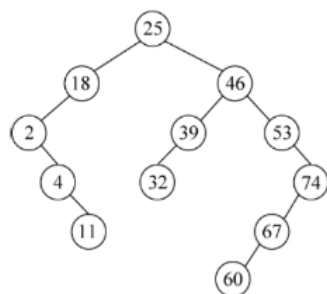
选择排序，选一个最小的放到左边

冒泡排序，把最大的冒到右边

等等....

25. 什么是对半搜索算法？适用条件是什么？算法复杂度如何？什么是二叉判定树，怎样利用二叉判定树计算平均搜索长度？

也就是二分查找，要求数组是有序的，复杂度是 $O(\log N)$ 复用一下上面的素材，二叉判定树也叫二叉搜索树



(a) 一棵二叉排序树

计算方法上面也有，注意如果是找不到的话（叶子结点）算到它的父节点就可以了

26. 什么是有向无环图DAG？它有怎样的特点？ DAG有什么性质？ DAG可以用来解决哪些问题？

和上面的AOV网有点相似，参考GPT，给出以下分析

DAG 的性质:

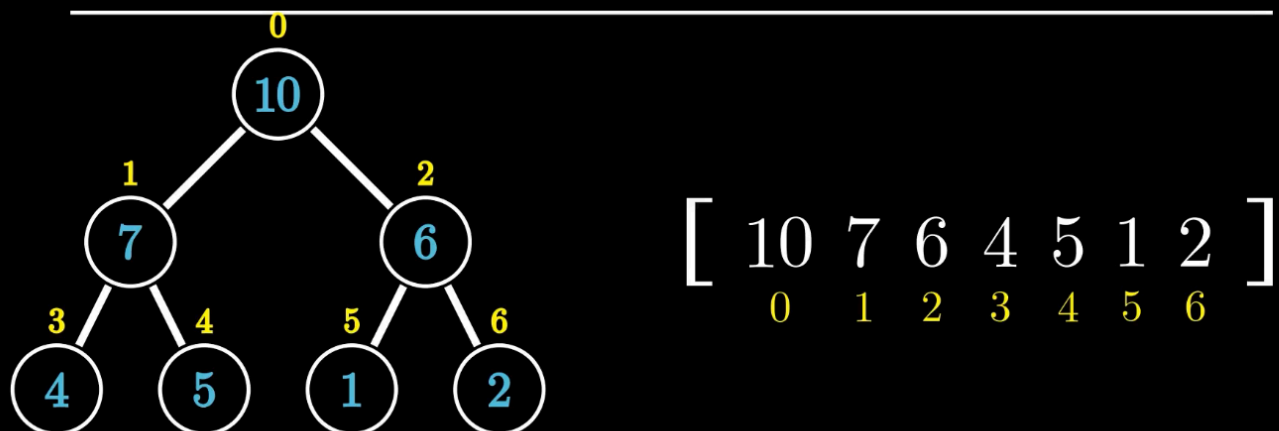
1. 拓扑排序 (**Topological Sorting**) : DAG 可以进行拓扑排序, 即将图中的所有节点排列成一个线性序列, 使得对于每一条有向边 $u \rightarrow v$, 节点 u 排在节点 v 之前。拓扑排序只适用于有向无环图。
 - 唯一性: 对于不同的 DAG, 可能有多个拓扑排序结果, 具体结果取决于图的结构。
 - 必要性: DAG 中总能找到至少一个拓扑排序。
2. 没有环: DAG 中的节点不存在自回路或环路, 即没有任何一个节点可以通过一系列有向边重新回到自身。
3. 最短路径问题: 在 DAG 中, 计算从某个节点到其他节点的最短路径是比较高效的, 因为不存在环路, 所以可以使用拓扑排序结合动态规划来求解最短路径问题。
4. 生成树: DAG 的有向边集具有生成树性质, 常用于解决最小生成树或最大生成树的问题。
5. 层次结构: DAG 的节点可以表示层次结构, 具有父子关系。特别适合用于表示任务的依赖关系、工作流的调度等。

27. 什么是堆 (heap) ? 堆有什么性质? 如何构建堆?

堆是一种基于完全二叉树的存储结构, 常见为最大/最小堆, 描述其中父节点和子节点的大小关系

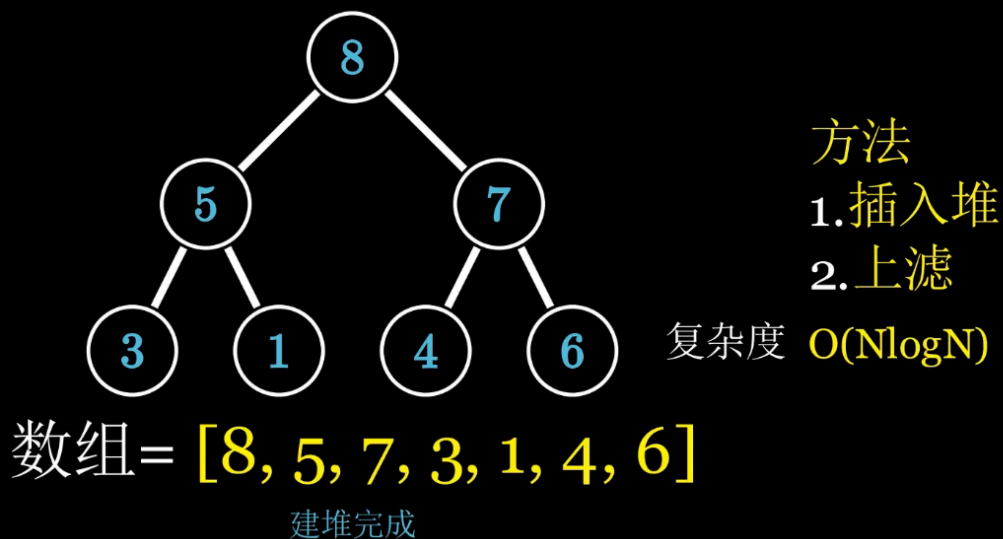
从一个堆, 我们采用层次遍历的方法可以还原数组, 如下

堆的储存



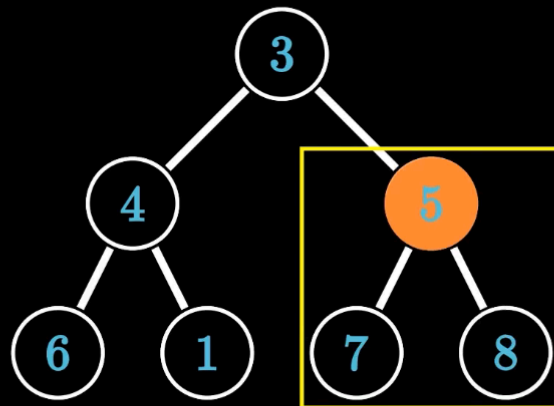
使用上，下滤的方式，我们可以快速从一个完全二叉树构建出堆
我们通常从数组构建堆，也是两种方法，简单图示如下

自顶向下建堆法



从下到上

自下而上建堆法

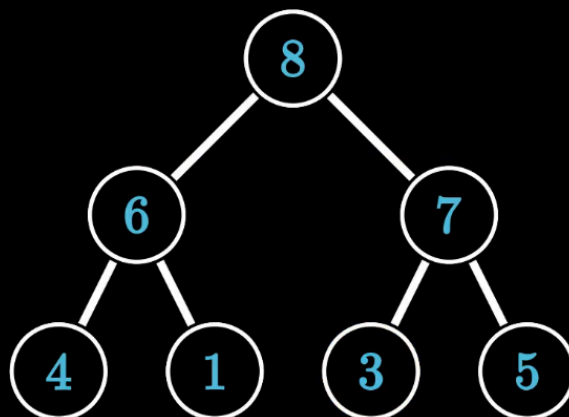


方法
对每个父结点进行下滤

数组 = [3, 4, 5, 6, 1, 7, 8]

对每一个父节点进行下滤操作

自下而上建堆法



方法
对每个父结点进行下滤

数组 = [3, 4, 5, 6, 1, 7, 8]

这也是堆排序的一个思路

构建一个小根堆，弹出根节点，把最后一个节点放到根部，再逐级下滤

